

Emergency Call Dispatch Queue

การวิเคราะห์ปัญหา: Emergency Call Dispatch Queue

1. Problem Statement

ศูนย์รับแจ้งเหตุฉุกเฉินได้รับแจ้งเหตุหลายรายการเข้ามาอย่างต่อเนื่อง เช่น อุบัติเหตุ ไฟไหม้ เหตุเจ็บป่วย และเหตุทั่วไป

โดยแต่ละเหตุการณ์มีระดับความรุนแรงแตกต่างกัน

ปัญหา คือ หากจัดการเหตุการณ์ตามลำดับเวลาที่รับแจ้งเพียงอย่างเดียว เหตุการณ์ที่มีความรุนแรงสูงอาจต้อง
เหตุการณ์ที่มีความรุนแรงต่ำกว่า ดังนั้นจึงต้องออกแบบระบบ Queue เพื่อจัดลำดับการประมวลผลเหตุการณ์
และเปรียบเทียบ 2 วิธี ได้แก่ FIFO Queue และ Priority Queue

2. Input

ข้อมูลที่ระบบได้รับการแจ้งเหตุ ประกอบด้วย

Attribute	รายละเอียด
Event ID	รหัสเหตุการณ์
Event Type	ประเภทของเหตุการณ์
Time	เวลาที่ได้รับแจ้ง
Severity	ระดับความรุนแรง
Location	สถานที่เกิดเหตุ

ตัวอย่างระดับความรุนแรง

Severity	Priority
Critical	1
High	2
Medium	3
Low	4

หมายเหตุ: ตัวเลข Priority ยิ่งน้อย หมายถึงความสำคัญยิ่งสูง

3. Output

ระบบต้องแสดงลำดับการประมวลผล Event ที่ออกจาก Queue พร้อมข้อมูลสำคัญ ได้แก่

- Event ID
- Event Type
- Severity
- Location
- ลำดับการประมวลผล

และเปรียบเทียบผลลัพธ์ระหว่าง

- FIFO Event Queue
- Priority Event Queue

4. Constraints

1. Event ID ต้องไม่ซ้ำกัน
2. Event แต่ละรายการต้องมีข้อมูลครบถ้วน
3. Severity มี 4 ระดับ ได้แก่ Critical, High, Medium และ Low
4. FIFO Queue ต้องประมวลผลตามลำดับเวลาที่รับแจ้ง
5. Priority Queue ต้องประมวลผลเหตุการณ์ที่มีความรุนแรงสูงก่อน
6. หาก Event มี Priority เท่ากัน ให้พิจารณาลำดับเวลาที่รับแจ้ง โดย Event ที่เข้าก่อนต้องได้รับการประมวลผลก่อน
7. ระบบต้องรองรับกรณี Queue ว่าง
8. ระบบต้องสามารถยกเลิก Event ที่อยู่ใน Queue ได้

5. Queue เก็บข้อมูลอะไร

Queue จะเก็บข้อมูลประเภท **Event** โดยแต่ละ Event ประกอบด้วย

Event ID, Event Type, Time, Severity, Location

ตัวอย่าง

E1 | เจ็บป่วย | 10:01 | Medium | อาคาร A

E2 | เหตุทั่วไป | 10:02 | Low | ถนน B

E3 | อุบัติเหตุ | 10:03 | Critical | ถนน C

6. Queue ชนิดใดเหมาะสม

Algorithm A: FIFO Queue

ใช้ `ArrayDeque<Event>` เพราะต้องการจัดการเหตุการณ์ตามลำดับที่เข้ามาก่อน-หลัง

$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4$

จะประมวลผลเป็น

$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4$

Algorithm B: Priority Queue

ใช้ `PriorityQueue<Event>` เพราะต้องการจัดลำดับเหตุการณ์ตามระดับความรุนแรง

$\text{Critical} \rightarrow \text{High} \rightarrow \text{Medium} \rightarrow \text{Low}$

ตัวอย่าง

E1 Medium

E2 Low

E3 Critical

E4 High

จะถูกประมวลผลเป็น

$E3 \rightarrow E4 \rightarrow E1 \rightarrow E2$

7. เหตุผลที่ต้องใช้ Queue

ระบบ Emergency Call Dispatch มีลักษณะเป็นข้อมูลที่เข้ามาอย่างต่อเนื่องและมีเหตุการณ์จำนวนมากรอการประมวลผล จึงเหมาะกับโครงสร้างข้อมูลแบบ Queue

FIFO Queue เหมาะกับกรณีที่ต้องการความยุติธรรมตามลำดับเวลาที่แจ้งเหตุ ส่วน **Priority Queue** เหมาะกับ

สถานการณ์ฉุกเฉินจริง เพราะสามารถให้เหตุการณ์ที่มีความรุนแรงสูงได้รับการจัดการก่อนเหตุการณ์ทั่วไป

ดังนั้น การเปรียบเทียบทั้งสอง Algorithm จะช่วยให้สามารถเลือกโครงสร้างข้อมูลที่เหมาะสมกับระบบ Emergency

Call Dispatch ได้อย่างมีประสิทธิภาพ

Queue Design

การออกแบบ Queue สำหรับระบบ Emergency Call Dispatch

ระบบ Emergency Call Dispatch ออกแบบ Queue จำนวน 2 รูปแบบ เพื่อเปรียบเทียบประสิทธิภาพและความเหมาะสมในการจัดลำดับเหตุการณ์ฉุกเฉิน ได้แก่ FIFO Event Queue และ Priority Event Queue

1. ข้อมูลที่จัดเก็บใน Queue

ระบบกำหนดข้อมูลของแต่ละเหตุการณ์เป็น Event โดยประกอบด้วยข้อมูลดังนี้

Attribute	Data Type	รายละเอียด
Event ID	String	รหัสประจำเหตุการณ์
Event Type	String	ประเภทของเหตุการณ์
Time	String	เวลาที่รับแจ้งเหตุ
Severity	int	ระดับความรุนแรง
Location	String	สถานที่เกิดเหตุ
Sequence	int	ลำดับการเข้ามาของ Event

ระดับความรุนแรง

Severity	Priority	ความหมาย
Critical	1	เหตุฉุกเฉินร้ายแรงมาก
High	2	เหตุร้ายแรง
Medium	3	เหตุระดับปานกลาง
Low	4	เหตุทั่วไป

หลักการ: ค่า Priority ยิ่งน้อย หมายถึงความสำคัญยิ่งสูง

2. Queue Design แบบที่ 1: FIFO Event Queue Data Structure

ใช้

```
Queue<Event> queue = new ArrayDeque<>();
```

หรือกำหนดเป็น

```
ArrayDeque<Event> queue = new ArrayDeque<>();
```

หลักการทำงาน

ใช้หลัก **First In First Out (FIFO)** หมายถึง Event ที่เข้ามาก่อนจะได้รับการประมวลผลก่อน โดยไม่พิจารณาระดับความรุนแรง

ตัวอย่าง

ENQUEUE

$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4 \rightarrow E5 \rightarrow E6$

↑

Front

เมื่อเรียก dequeue() จะได้

$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4 \rightarrow E5 \rightarrow E6$

↑

นำออกก่อน

ดังนั้นลำดับการประมวลผลคือ

$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4 \rightarrow E5 \rightarrow E6$

ข้อดี

- เข้าใจง่าย
- Implement ได้ง่าย
- enqueue() และ dequeue() มีประสิทธิภาพ $O(1)$
- มีความยุติธรรมตามลำดับเวลาที่แจ้ง

ข้อจำกัด

- ไม่พิจารณาความรุนแรงของเหตุการณ์
- เหตุการณ์ Critical อาจต้องรอเหตุการณ์ Low ที่เข้ามาก่อน

3. Queue Design แบบที่ 2: Priority Event Queue Data Structure

ใช้

```
PriorityQueue<Event> queue;
```

โดยกำหนด Comparator เพื่อจัดลำดับตาม Severity

```
PriorityQueue<Event> queue =
```

```
    new PriorityQueue<>(  
        Comparator.comparingInt(Event::getSeverity)
```

```
            .thenComparingInt(Event::getSequence)
```

```
    );
```

หลักการทำงาน

Priority Queue จะเลือก Event ที่มี Priority สูงที่สุดออกมาก่อน

กำหนดลำดับ

Critical = 1

High = 2

Medium = 3

Low = 4

ดังนั้น

1 → 2 → 3 → 4

หมายถึง

Critical → High → Medium → Low

ตัวอย่าง

กำหนด Event ดังนี้

E1 = Medium

E2 = Low

E3 = Critical

E4 = High

E5 = Critical

E6 = Medium

เมื่อใส่ทั้งหมดเข้า Priority Queue ระบบจะจัดลำดับตาม Priority เป็น

E3 → E5 → E4 → E1 → E6 → E2

โดย

E3 = Critical = 1

E5 = Critical = 1

E4 = High = 2

E1 = Medium = 3

E6 = Medium = 3

E2 = Low = 4

4. การจัดการกรณี Priority เท่ากัน

กรณีที่ Event มี Priority เท่ากัน ระบบจะใช้ **Sequence** เป็นตัวตัดสิน

ตัวอย่าง

E3 = Critical = 1

E5 = Critical = 1

เนื่องจาก

E3 เข้ามาก่อน E5

จึงต้องประมวลผล

E3 → E5

ไม่ใช่

E5 → E3

ดังนั้น Comparator จึงมี 2 เงื่อนไข

```
Comparator.comparingInt(Event::getSeverity)  
                .thenComparingInt(Event::getSequence)
```

ความหมายคือ

1. เปรียบเทียบ Severity ก่อน
2. ถ้า Severity เท่ากัน ให้เปรียบเทียบ Sequence

5. Queue Operations

ระบบออกแบบให้รองรับ Operation หลักดังนี้

Operation	ความหมาย
enqueue() / offer()	เพิ่ม Event เข้า Queue
dequeue() / poll()	นำ Event ออกจาก Queue
peek()	ดู Event ที่อยู่ด้านหน้าโดยไม่ลบ
search()	ค้นหา Event
display()	แสดง Event ทั้งหมด
cancel()	ยกเลิก Event ที่อยู่ใน Queue

6. การออกแบบ Cancel Event

ระบบต้องรองรับการยกเลิกเหตุการณ์ที่อยู่ใน Queue

กรณี Event อยู่ใน Queue

ก่อนยกเลิก

[E1, E2, E3, E4, E5]

ยกเลิก E3

หลังยกเลิก

[E1, E2, E4, E5]

กรณี Event ไม่มีอยู่ใน Queue

เช่น ยกเลิก

E99

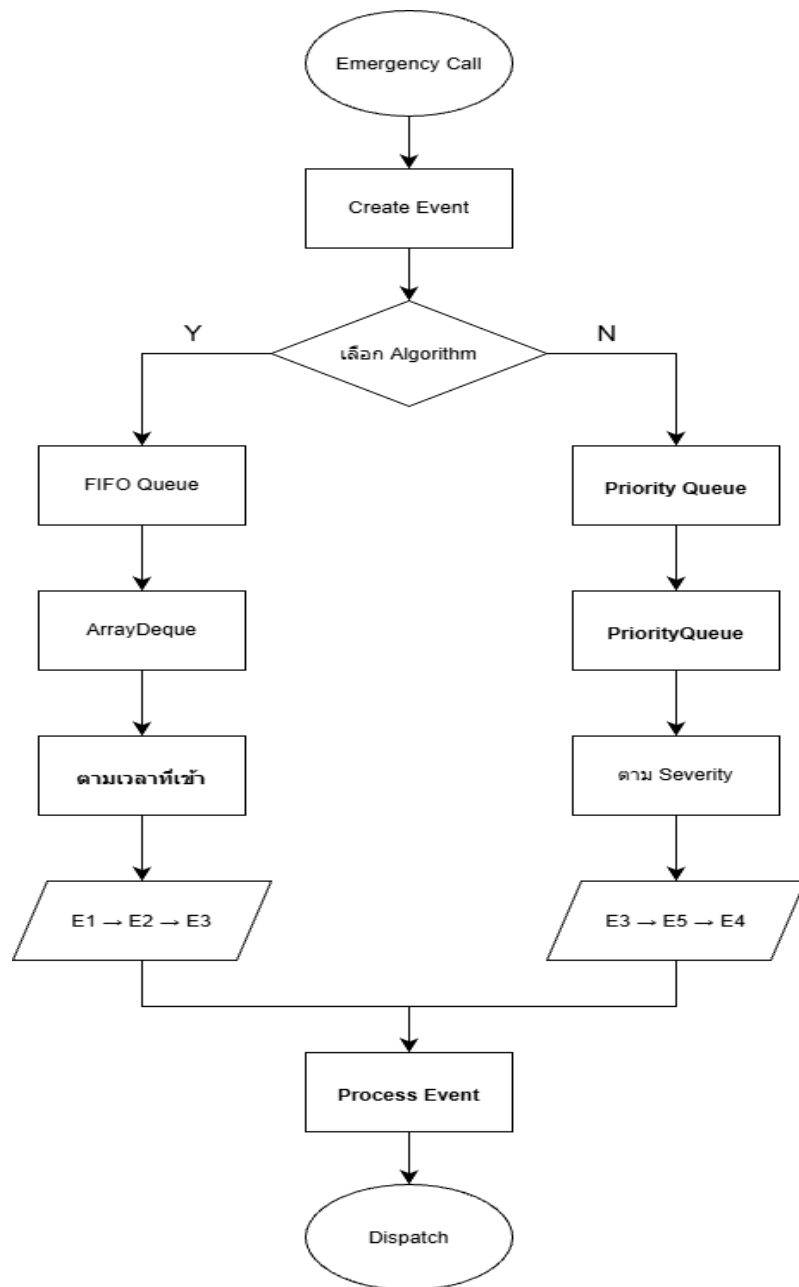
แต่ไม่มี E99 อยู่ใน Queue

ระบบจะแสดง

ไม่พบ Event E99 ใน Queue

7. ภาพรวม Queue Design

สามารถเขียนเป็น Flow ได้ดังนี้



8. เปรียบเทียบ Queue Design

ประเด็น	FIFO Event Queue	Priority Event Queue
Data Structure	ArrayDeque	PriorityQueue
หลักการ	เข้าก่อนออกก่อน	Priority สูงออกก่อน
พิจารณา Severity	ไม่พิจารณา	พิจารณา
พิจารณาเวลา	พิจารณา	ใช้เมื่อ Priority เท่ากัน
Enqueue	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(\log n)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
Space	$O(n)$	$O(n)$
ความยุติธรรมตามเวลา	สูง	ปานกลาง
เหมาะกับ	เหตุทั่วไป	เหตุฉุกเฉิน

9. Design Decision

สำหรับระบบ **Emergency Call Dispatch** ทั้งสองโครงสร้างสามารถใช้งานได้ แต่ **Priority Queue** เหมาะสมกับระบบจริงมากกว่า เพราะเหตุการณ์ฉุกเฉินไม่ควรได้รับการจัดการเฉพาะตามเวลาที่แจ้ง แต่ควรพิจารณาความรุนแรงร่วมด้วย

ตัวอย่างเช่น หากมี

E1 = Low

E2 = Critical

และ E1 เข้ามาก่อน E2

FIFO

E1 → E2

แต่ **Priority Queue**

E2 → E1

ซึ่งเหมาะสมกว่าในสถานการณ์ที่ต้องให้ความสำคัญกับชีวิตและความปลอดภัย

Pseudocode Algorithm A

Algorithm A: FIFO Event Queue

หลักการ: ประมวลผล Event ตามลำดับเวลาที่รับแจ้งเข้ามา โดย Event ที่เข้าก่อนจะถูกประมวลผลก่อน

Pseudocode

ALGORITHM FIFO_EVENT_QUEUE

START

Create empty Queue

FOR each Event

 ENQUEUE Event into Queue

END FOR

WHILE Queue is not empty

 Event $\leftarrow$ DEQUEUE from Queue

 PROCESS Event

END WHILE

END

การทำงาน

ตัวอย่าง Event

$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4 \rightarrow E5 \rightarrow E6$

เมื่อใช้ FIFO จะประมวลผลตามลำดับ

$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4 \rightarrow E5 \rightarrow E6$

โดยไม่สนใจว่า Event มี Severity ระดับใด

Pseudocode Algorithm B

Algorithm B: Priority Event Queue

หลักการ: ประมวลผล Event ที่มีระดับความรุนแรงสูงก่อน โดย Critical มี Priority สูงสุด และหาก Priority เท่ากัน ให้ Event ที่เข้ามาก่อนถูกประมวลผลก่อน

Pseudocode

ALGORITHM PRIORITY_EVENT_QUEUE

START

Create empty Priority Queue

FOR each Event

 INSERT Event into Priority Queue

END FOR

WHILE Priority Queue is not empty

 Event $\leftarrow$ REMOVE highest-priority Event

PROCESS Event

END WHILE

END

Priority ที่กำหนด

Critical = 1

High = 2

Medium = 3

Low = 4

หมายเหตุ: เลข Priority ยิ่งน้อย หมายถึงความสำคัญยิ่งสูง

สรุปเปรียบเทียบ Pseudocode

	Algorithm A: FIFO	Algorithm B: Priority
โครงสร้าง	Queue	Priority Queue
หลักการ	เข้าก่อนออกก่อน	Priority สูงออกก่อน
พิจารณาเวลา	✓	✓ กรณี Priority เท่ากัน
พิจารณา Severity	✗	✓
ตัวอย่างผลลัพธ์	E1 → E2 → E3 → E4...	E3 → E5 → E4 → E1...

Queue Trace

ข้อมูล Event

Event	Severity	Priority
E1	Medium	3
E2	Low	4
E3	Critical	1
E4	High	2
E5	Critical	1
E6	Medium	3

Queue Trace — Algorithm A: FIFO Event Queue

หลักการ: Event ที่เข้ามาก่อนจะถูกประมวลผลก่อน โดยไม่พิจารณา Severity

Step	Operation	Queue Before	Queue After	Output
1	ENQUEUE E1	[]	[E1]	-
2	ENQUEUE E2	[E1]	[E1, E2]	-
3	ENQUEUE E3	[E1, E2]	[E1, E2, E3]	-
4	ENQUEUE E4	[E1, E2, E3]	[E1, E2, E3, E4]	-
5	ENQUEUE E5	[E1, E2, E3, E4]	[E1, E2, E3, E4, E5]	-
6	ENQUEUE E6	[E1, E2, E3, E4, E5]	[E1, E2, E3, E4, E5, E6]	-
7	DEQUEUE	[E1, E2, E3, E4, E5, E6]	[E2, E3, E4, E5, E6]	E1
8	DEQUEUE	[E2, E3, E4, E5, E6]	[E3, E4, E5, E6]	E2
9	DEQUEUE	[E3, E4, E5, E6]	[E4, E5, E6]	E3
10	DEQUEUE	[E4, E5, E6]	[E5, E6]	E4
11	DEQUEUE	[E5, E6]	[E6]	E5
12	DEQUEUE	[E6]	[]	E6

ผลลัพธ์

E1 → E2 → E3 → E4 → E5 → E6

อธิบาย

ใน Step 1–6 Event ถูกเพิ่มเข้า Queue ตามลำดับเวลาที่ได้รับแจ้ง จึงเรียงเป็น E1 ถึง E6

เมื่อเริ่ม DEQUEUE ระบบจะนำ Event ที่อยู่ด้านหน้า Queue ออกก่อนเสมอ ดังนั้น E1 ซึ่งเข้ามาก่อนจึงถูกนำออกก่อน ตามด้วย E2, E3, E4, E5 และ E6

Queue Trace — Algorithm B: Priority Event Queue

หลักการ: Event ที่มี Priority สูงกว่าจะถูกประมวลผลก่อน

Critical = 1

High = 2

Medium = 3

Low = 4

กรณี Priority เท่ากัน ให้ Event ที่เข้ามาก่อนถูกประมวลผลก่อน

Step	Operation	Queue Before	Queue After	Output
1	ENQUEUE E1	[]	[E1]	-
2	ENQUEUE E2	[E1]	[E1, E2]	-
3	ENQUEUE E3	[E1, E2]	[E3, E1, E2]	-
4	ENQUEUE E4	[E3, E1, E2]	[E3, E4, E1, E2]	-
5	ENQUEUE E5	[E3, E4, E1, E2]	[E3, E5, E4, E1, E2]	-
6	ENQUEUE E6	[E3, E5, E4, E1, E2]	[E3, E5, E4, E1, E6, E2]	-
7	DEQUEUE	[E3, E5, E4, E1, E6, E2]	[E5, E4, E1, E6, E2]	E3
8	DEQUEUE	[E5, E4, E1, E6, E2]	[E4, E1, E6, E2]	E5
9	DEQUEUE	[E4, E1, E6, E2]	[E1, E6, E2]	E4
10	DEQUEUE	[E1, E6, E2]	[E6, E2]	E1

11	DEQUEUE	[E6, E2]	[E2]	E6
12	DEQUEUE	[E2]	[]	E2

ผลลัพธ์

$E3 \rightarrow E5 \rightarrow E4 \rightarrow E1 \rightarrow E6 \rightarrow E2$

อธิบาย

หลังจากเพิ่ม Event ครบทั้งหมด ระบบจะพิจารณา Severity ของแต่ละ Event

$E3 = \text{Critical} = 1$

$E5 = \text{Critical} = 1$

$E4 = \text{High} = 2$

$E1 = \text{Medium} = 3$

$E6 = \text{Medium} = 3$

$E2 = \text{Low} = 4$

ดังนั้น E3 และ E5 ซึ่งเป็น Critical จะถูกนำออกก่อน ตามด้วย E4 ที่เป็น High จากนั้น E1 และ E6 ที่เป็น Medium และสุดท้าย E2 ที่เป็น Low

กรณี E3 และ E5 มี Priority เท่ากัน ระบบใช้ลำดับการเข้ามาเป็นตัวตัดสิน ทำให้ E3 ออกก่อน E5

เปรียบเทียบผลการ Trace

ลำดับ	FIFO	Priority Queue
1	E1	E3
2	E2	E5
3	E3	E4
4	E4	E1
5	E5	E6
6	E6	E2

สรุป

FIFO

E1 → E2 → E3 → E4 → E5 → E6

เน้น ลำดับเวลาที่รับแจ้ง

Priority Queue

E3 → E5 → E4 → E1 → E6 → E2

เน้น ระดับความรุนแรงของเหตุการณ์

ดังนั้น Priority Queue เหมาะกับระบบ Emergency Call Dispatch มากกว่า เพราะสามารถให้เหตุการณ์

Critical และ High ได้รับการประมวลผลก่อน แม้จะเข้ามาทีหลังเหตุการณ์ที่มีความรุนแรงต่ำกว่า

Correctness / Invariant

Algorithm A: FIFO Event Queue

Correctness

Algorithm A ทำงานถูกต้องตามหลัก **FIFO (First In First Out)** โดย Event ที่เข้ามาก่อนจะถูกนำออกจาก Queue และประมวลผลก่อน Event ที่เข้ามาทีหลัง

เมื่อใช้ enqueue() Event ใหม่จะถูกเพิ่มเข้าไปที่ด้านท้ายของ Queue และเมื่อใช้ dequeue() ระบบจะนำ Event ที่อยู่ด้านหน้าออกไปประมวลผล

Invariant

ก่อนเริ่มการประมวลผลในแต่ละรอบ สมาชิกที่อยู่บริเวณ Front ของ Queue คือ Event ที่เข้ามาก่อน Event อื่นทั้งหมดที่ยังไม่ได้รับการประมวลผล

ตัวอย่าง

Queue = [E1, E2, E3, E4]

↑

Front

เมื่อ DEQUEUE

Output = E1

Queue = [E2, E3, E4]

↑

Front

ดังนั้น E1 ซึ่งเข้ามาก่อนจึงถูกประมวลผลก่อน E2, E3 และ E4

สรุปความถูกต้อง

ตราบใดที่ Queue ไม่ว่าง การ dequeue() จะนำสมาชิกที่อยู่ Front ออกเสมอ ทำให้ลำดับการประมวลผลเป็นไปตามลำดับที่ Event เข้ามา จึงถูกต้องตามหลัก FIFO

Algorithm B: Priority Event Queue

Correctness

Algorithm B ทำงานถูกต้องตามหลัก **Priority Queue** โดย Event ที่มีระดับความรุนแรงสูงกว่าจะได้รับการประมวลผลก่อน

กำหนด Priority ดังนี้

Critical = 1

High = 2

Medium = 3

Low = 4

ดังนั้น Priority ที่มีค่าตัวเลขน้อยกว่าจะได้รับการประมวลผลก่อน

Invariant

ก่อนเริ่มการประมวลผลในแต่ละรอบ Event ที่ถูกเลือกจาก Priority Queue จะเป็น Event ที่มี Priority สูงที่สุดในบรรดา Event ที่ยังไม่ได้รับการประมวลผล

ตัวอย่าง

Queue

E1 = Medium = 3

E2 = Low = 4

E3 = Critical = 1

E4 = High = 2

เมื่อเลือก Event ออก ระบบจะเลือก

E3 = Critical = 1

ก่อน เพราะมี Priority สูงที่สุด

หลังจากนำ E3 ออก

E4 = High = 2

E1 = Medium = 3

E2 = Low = 4

ดังนั้น Event ถัดไปคือ E4

กรณี Priority เท่ากัน

หาก Event มี Priority เท่ากัน ให้ใช้ **Sequence** หรือเวลาที่รับแจ้ง เป็นตัวตัดสิน

ตัวอย่าง

$E3 = \text{Critical} = 1$

$E5 = \text{Critical} = 1$

เนื่องจาก $E3$ เข้ามาก่อน $E5$

$E3 \rightarrow E5$

จึงถูกต้องตามเงื่อนไขของระบบ

สรุปความถูกต้อง

ทราบใดที่ Priority Queue ไม่ว่าง ระบบจะเลือก Event ที่มี Priority สูงที่สุดออกมาประมวลผลก่อนเสมอ และหากมี Priority เท่ากัน จะเลือก Event ที่เข้ามาก่อน ทำให้ลำดับการประมวลผลถูกต้องตามระดับความรุนแรง และลำดับเวลาที่รับแจ้ง

สรุป Correctness / Invariant

ประเด็น	Algorithm A: FIFO	Algorithm B: Priority
หลักการ	เข้าก่อนออกก่อน	Priority สูงออกก่อน
Invariant	Front คือ Event ที่เข้ามาก่อนที่สุด	Front คือ Event ที่มี Priority สูงที่สุด
กรณี Priority เท่ากัน	ไม่เกี่ยวข้อง	ใช้ลำดับเวลาที่เข้ามา
ความถูกต้อง	รักษาลำดับการเข้ามา	รักษาลำดับตามความรุนแรง
ผลลัพธ์ตัวอย่าง	$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4 \rightarrow E5 \rightarrow E6$	$E3 \rightarrow E5 \rightarrow E4 \rightarrow E1 \rightarrow E6 \rightarrow E2$

Algorithm A ถูกต้องเนื่องจากรักษา Invariant ของ FIFO โดย Event ที่อยู่บริเวณ Front จะเป็น Event ที่เข้ามาก่อน Event อื่นที่ยังไม่ได้ประมวลผล จึงทำให้ Event ถูกประมวลผลตามลำดับเวลา

Algorithm B ถูกต้องเนื่องจากรักษา Invariant ของ Priority Queue โดย Event ที่มี Priority สูงที่สุดจะถูกเลือกออกมาก่อนเสมอ และหากมี Priority เท่ากัน จะพิจารณาลำดับเวลาที่รับแจ้ง เพื่อให้ Event ที่เข้ามาก่อนได้รับการประมวลผลก่อน

Time Complexity

ในการวิเคราะห์ Time Complexity ของระบบ **Emergency Call Dispatch Queue** จะพิจารณาการทำงานหลัก ได้แก่ enqueue(), dequeue(), peek(), search() และ display()

Algorithm A: FIFO Event Queue

Algorithm A ใช้ ArrayDeque<Event> และทำงานตามหลัก FIFO

Operation	Method	Time Complexity
เพิ่ม Event	offer() / enqueue()	O(1)
นำ Event ออก	poll() / dequeue()	O(1)
ดู Event ด้านหน้า	peek()	O(1)
ค้นหา Event	search()	O(n)
แสดง Event ทั้งหมด	display()	O(n)

อธิบาย

1. Enqueue — O(1)

การเพิ่ม Event เข้าไปท้าย Queue สามารถทำได้ในเวลาคงที่

$[E1, E2] \rightarrow \text{ENQUEUE } E3 \rightarrow [E1, E2, E3]$

จึงมี Time Complexity เป็น O(1)

2. Dequeue — $O(1)$

การนำ Event ด้านหน้าออกสามารถทำได้โดยไม่ต้องเลื่อนสมาชิกทั้งหมด

[E1, E2, E3]

↓ DEQUEUE

[E2, E3]

จึงเป็น $O(1)$

3. Peek — $O(1)$

การดู Event ที่อยู่ด้านหน้าไม่ต้องค้นหาข้อมูลทั้งหมด

จึงเป็น $O(1)$

4. Search — $O(n)$

การค้นหา Event เช่น E5 อาจต้องตรวจสอบตั้งแต่ Event ตัวแรกจนถึงตัวสุดท้าย

E1 → E2 → E3 → E4 → E5

↑

พบข้อมูล

กรณีแย่ที่สุดต้องตรวจสอบทั้งหมด n รายการ จึงเป็น $O(n)$

5. Display — $O(n)$

การแสดงผล Event ทั้งหมดต้องเข้าถึงสมาชิกทุกตัว จึงเป็น $O(n)$

Algorithm B: Priority Event Queue

Algorithm B ใช้ PriorityQueue<Event> เพื่อจัดลำดับตาม Severity

Operation	Method	Time Complexity
เพิ่ม Event	offer()	$O(\log n)$
นำ Event ออก	poll()	$O(\log n)$
ดู Event ที่ Priority สูงสุด	peek()	$O(1)$
ค้นหา Event	search()	$O(n)$
แสดง Event ทั้งหมด	display()	$O(n)$

อธิบาย

1. Enqueue / Offer — $O(\log n)$

เมื่อเพิ่ม Event เข้า PriorityQueue ระบบต้องปรับโครงสร้าง Heap เพื่อรักษาลำดับ Priority

ENQUEUE E3



ตรวจสอบ Priority



ปรับตำแหน่งใน Heap

จึงมี Time Complexity เป็น $O(\log n)$

2. Dequeue / Poll — $O(\log n)$

เมื่อ `poll()` Event ที่มี Priority สูงสุดออก ระบบต้องปรับ Heap ใหม่

ตัวอย่าง

E3(Critical)

↓ `poll()`

E5(Critical)

E4(High)

E1(Medium)

จึงมี Time Complexity เป็น $O(\log n)$

3. Peek — $O(1)$

`peek()` สามารถดู Event ที่มี Priority สูงสุดได้โดยไม่ต้องนำออกจาก Queue

จึงเป็น $O(1)$

4. Search — $O(n)$

การค้นหา Event เช่น E5 ไม่สามารถใช้ Priority เพื่อค้นหา Event โดยตรงได้ จึงอาจต้องตรวจสอบสมาชิกทั้งหมด

ดังนั้นเป็น $O(n)$

5. Display — $O(n)$

หากต้องการแสดง Event ทั้งหมด ต้องเข้าถึง Event ทุกตัว จึงเป็น $O(n)$

หมายเหตุ: การ iterate PriorityQueue ของ Java ไม่ได้หมายความว่าเราได้ลำดับ Priority ที่เรียงสมบูรณ์ ดังนั้นถ้าต้องการแสดง ลำดับการประมวลผลจริง ควรใช้ `poll()` ทีละตัว

เปรียบเทียบ Time Complexity

Operation	FIFO ArrayDeque	Priority PriorityQueue
enqueue() / offer()	$O(1)$	$O(\log n)$
dequeue() / poll()	$O(1)$	$O(\log n)$
peek()	$O(1)$	$O(1)$
search()	$O(n)$	$O(n)$
display()	$O(n)$	$O(n)$

5.4 วิเคราะห์โดยรวม

FIFO Queue มีประสิทธิภาพสูงกว่าในด้านการเพิ่มและนำ Event ออกจาก Queue เนื่องจาก enqueue() และ dequeue() ใช้เวลา $O(1)$

ส่วน **Priority Queue** มีค่าใช้จ่ายมากกว่า เนื่องจากการ offer() และ poll() ต้องรักษาลำดับของ Heap จึงใช้เวลา $O(\log n)$

อย่างไรก็ตาม Priority Queue มีข้อได้เปรียบที่สำคัญสำหรับระบบ Emergency Call Dispatch คือสามารถเลือก Event ที่มี **Severity สูงที่สุด** ได้ทันทีด้วย peek() หรือ poll() ซึ่งเหมาะกับระบบฉุกเฉินที่ต้องให้ความสำคัญกับเหตุการณ์รุนแรงก่อน

Algorithm A: FIFO Event Queue มี enqueue() และ dequeue() เป็น $O(1)$ จึงเหมาะกับระบบที่ต้องการประมวลผล Event ตามลำดับเวลาและต้องการประสิทธิภาพสูง

Algorithm B: Priority Event Queue มี offer() และ poll() เป็น $O(\log n)$ และ peek() เป็น $O(1)$ เนื่องจากใช้ Heap ในการจัดลำดับ Priority แต่เหมาะกับระบบ Emergency Call Dispatch มากกว่า เพราะสามารถประมวลผลเหตุการณ์ที่มีความรุนแรงสูงก่อนเหตุการณ์ทั่วไปได้

Space Complexity

ระบบ **Emergency Call Dispatch Queue** มีการจัดเก็บข้อมูล Event ที่รอการประมวลผล โดยกำหนดให้มี Event สูงสุดจำนวน n รายการ

แต่ละ Event ประกอบด้วย

Event ID, Event Type, Time, Severity, Location, Sequence

ดังนั้นพื้นที่ที่ใช้ในการจัดเก็บ Event จะเพิ่มขึ้นตามจำนวน Event ที่อยู่ใน Queue

Algorithm A: FIFO Event Queue

Algorithm A ใช้

```
Queue<Event> queue = new ArrayDeque<>();
```

หากมี Event จำนวน n รายการ Queue จะต้องใช้พื้นที่สำหรับเก็บ Event ทั้งหมด

ดังนั้น

Space Complexity = $O(n)$

ตัวอย่าง

ถ้ามี Event

$n = 6$

[E1, E2, E3, E4, E5, E6]

Queue ต้องเก็บข้อมูลทั้ง 6 Event

ถ้าจำนวน Event เพิ่มขึ้น

$n = 100$

$n = 1,000$

$n = 10,000$

$n = 50,000$

พื้นที่ที่ใช้ก็จะเพิ่มขึ้นตามจำนวน Event

Algorithm B: Priority Event Queue

Algorithm B ใช้

PriorityQueue<Event> queue;

เช่นเดียวกับ FIFO Queue หากมี Event จำนวน n รายการ ระบบต้องเก็บ Event ทั้งหมดไว้ใน Priority Queue

ดังนั้น

Space Complexity = $O(n)$

นอกจากนี้ PriorityQueue ต้องใช้พื้นที่สำหรับโครงสร้าง Heap เพื่อจัดลำดับ Priority แต่พื้นที่ดังกล่าวยังมีขนาดสัมพันธ์กับจำนวนสมาชิกใน Queue จึงยังคงเป็น $O(n)$

เปรียบเทียบ Space Complexity

Algorithm	Data Structure	Space Complexity
Algorithm A: FIFO	ArrayDeque<Event>	$O(n)$
Algorithm B: Priority	PriorityQueue<Event>	$O(n)$

ทั้งสอง Algorithm มี Space Complexity เป็น $O(n)$ เพราะต้องจัดเก็บ Event สูงสุด n รายการ

6.4 Additional Space

นอกจาก Queue แล้ว ระบบยังมีพื้นที่สำหรับตัวแปรอื่น ๆ เช่น

Event object

ตัวแปรสำหรับวน Loop

ตัวแปรสำหรับ Sequence

ตัวแปรสำหรับ Severity

แต่ตัวแปรเหล่านี้มีจำนวนคงที่ ไม่ได้เพิ่มตามจำนวน Event

จึงมี Additional Space เป็น

$O(1)$

ดังนั้น Space Complexity โดยรวมยังคงเป็น

$O(n)$

Algorithm A: FIFO Event Queue ใช้ ArrayDeque ในการจัดเก็บ Event จำนวนสูงสุด n รายการ จึงมี Space Complexity เป็น $O(n)$

Algorithm B: Priority Event Queue ใช้ PriorityQueue ซึ่งจัดเก็บ Event และโครงสร้าง Heap สำหรับจัดลำดับ Priority โดยมี Space Complexity เป็น $O(n)$

แม้ Priority Queue จะมีโครงสร้าง Heap เพิ่มเข้ามา แต่พื้นที่ที่ใช้ยังแปรผันตามจำนวน Event ดังนั้นทั้งสอง Algorithm จึงมี Space Complexity เท่ากันคือ $O(n)$

Java Program ក្រុម 7 : Emergency Call Dispatch Queue

EmergencyCallDispatch.java

```
import java.util.*;

public class EmergencyCallDispatch {

    // =====

    // Event Class

    // =====

    static class Event {

        private String eventId;

        private String eventType;

        private String time;

        private int severity;

        private String location;

        private int sequence;

        public Event(String eventId, String eventType, String time,

            int severity, String location, int sequence) {

            this.eventId = eventId;

            this.eventType = eventType;

            this.time = time;

            this.severity = severity;
        }
    }
}
```

```
        this.location = location;

        this.sequence = sequence;

    }

    public String getEventId() {

        return eventId;

    }

    public String getEventType() {

        return eventType;

    }

    public String getTime() {

        return time;

    }

    public int getSeverity() {

        return severity;

    }

    public String getLocation() {

        return location;

    }

    public int getSequence() {

        return sequence;

    }

    public String getSeverityName() {
```



```
switch (severity) {  
  
    case 1:  
  
        return "Critical";  
  
    case 2:  
  
        return "High";  
  
    case 3:  
  
        return "Medium";  
  
    case 4:  
  
        return "Low";  
  
    default:  
  
        return "Unknown";  
  
    }  
  
}
```

@Override

```
public String toString() {  
  
    return eventId + " | "  
  
        + eventType + " | "  
  
        + time + " | "  
  
        + getSeverityName() + " | "  
  
        + location;  
  
}
```

```
}

// =====

// Algorithm A: FIFO Queue

// =====

public static void fifoAlgorithm(List<Event> events) {

    Queue<Event> queue = new ArrayDeque<>();

    System.out.println();

    System.out.println("=====");

    System.out.println("    ALGORITHM A: FIFO QUEUE");

    System.out.println("=====");

    // ENQUEUE

    for (Event event : events) {

        queue.offer(event);

    }

    System.out.println();

    System.out.println("Queue after ENQUEUE:");

    display(queue);

    // PEEK

    if (!queue.isEmpty()) {
```

```
        System.out.println();

        System.out.println("PEEK: " + queue.peek());

    }

    // SEARCH

    System.out.println();

    System.out.println("SEARCH E3:");

    searchEvent(queue, "E3");


    // DEQUEUE

    System.out.println();

    System.out.println("Processing Order:");

    while (!queue.isEmpty()) {

        Event event = queue.poll();

        System.out.println("DEQUEUE -> " + event);

    }

    System.out.println();

    System.out.println("Queue after processing: []");

}

//=====
```

```

// Algorithm B: Priority Queue

// =====

public static void priorityAlgorithm(List<Event> events) {

    PriorityQueue<Event> queue =

        new PriorityQueue<>(

            Comparator.comparingInt(Event::getSeverity)

                .thenComparingInt(Event::getSequence)

        );

    System.out.println();

    System.out.println("=====");

    System.out.println("  ALGORITHM B: PRIORITY QUEUE");

    System.out.println("=====");

    // ENQUEUE

    for (Event event : events) {

        queue.offer(event);

    }

    System.out.println();

    System.out.println("Queue after ENQUEUE:");

```

```
displayPriority(queue);

// PEEK

if (!queue.isEmpty()) {

    System.out.println();

    System.out.println("PEEK: " + queue.peek());

}

// SEARCH

System.out.println();

System.out.println("SEARCH E3:");

searchEvent(queue, "E3");

// DEQUEUE

System.out.println();

System.out.println("Processing Order:");

while (!queue.isEmpty()) {

    Event event = queue.poll();

    System.out.println("DEQUEUE -> " + event);

}
```

```
System.out.println();

System.out.println("Queue after processing: []");

}

// =====

// Search Event

// =====

public static boolean searchEvent(

    Collection<Event> queue, String eventId) {

    for (Event event : queue) {

        if (event.getEventId().equals(eventId)) {

            System.out.println("Event found: " + event);

            return true;

        }

    }

    System.out.println("Event not found: " + eventId);

    return false;

}
```

```
// =====  
  
// Cancel Event  
  
// =====  
  
public static boolean cancelEvent(  
    Queue<Event> queue, String eventId) {  
  
    Iterator<Event> iterator = queue.iterator();  
  
    while (iterator.hasNext()) {  
  
        Event event = iterator.next();  
  
        if (event.getEventId().equals(eventId)) {  
            iterator.remove();  
  
            System.out.println(  
                "Event cancelled successfully: " + eventId);  
  
            return true;  
        }  
    }  
}
```

```
System.out.println(

    "Event not found: " + eventId);

return false;

}

//=====

// Display FIFO Queue

//=====

public static void display(Collection<Event> queue) {

    System.out.println("[ ");

    for (Event event : queue) {

        System.out.println(" " + event);

    }

    System.out.println("]");

}

//=====

// Display Priority Queue
```



```
// =====

public static void displayPriority(
    PriorityQueue<Event> queue) {

    PriorityQueue<Event> copy =
        new PriorityQueue<>(queue);

    System.out.println("[ ");

    while (!copy.isEmpty()) {

        System.out.println(" " + copy.poll());

    }

    System.out.println("]");

}

// =====

// Main Method

// =====

public static void main(String[] args) {

    List<Event> events = new ArrayList<>();
```

```
// Event ID, Type, Time, Severity, Location, Sequence
```

```
events.add(new Event(  
    "E1", "Medical", "10:01",  
    3, "Building A", 1));
```

```
events.add(new Event(  
    "E2", "General Incident", "10:02",  
    4, "Road B", 2));
```

```
events.add(new Event(  
    "E3", "Accident", "10:03",  
    1, "Road C", 3));
```

```
events.add(new Event(  
    "E4", "Fire", "10:04",  
    2, "Building D", 4));
```

```
events.add(new Event(  
    "E5", "Accident", "10:05",  
    1, "Road E", 5));
```

```
events.add(new Event(  
  
    "E6", "Medical", "10:06",  
  
    3, "Building F", 6));
```

```
// =====
```

```
// Display All Events
```

```
// =====
```

```
System.out.println("=====");
```

```
System.out.println("  EMERGENCY CALL DISPATCH SYSTEM");
```

```
System.out.println("=====");
```

```
System.out.println();
```

```
System.out.println("Emergency Events:");
```

```
for (Event event : events) {
```

```
    System.out.println(event);
```

```
}
```

```
// =====
```

```
// Run Algorithm A
```

```
// =====
```

```
fifoAlgorithm(events);
```

```
// =====
```

```
// Run Algorithm B
```

```
// =====
```

```
priorityAlgorithm(events);
```

```
// =====
```

```
// Cancel Test
```

```
// =====
```

```
System.out.println();
```

```
System.out.println("=====");
```

```
System.out.println("    CANCEL EVENT TEST");
```

```
System.out.println("=====");
```

```
Queue<Event> cancelQueue = new ArrayDeque<>();
```

```
for (Event event : events) {
```

```
cancelQueue.offer(event);

}

System.out.println();

System.out.println("Queue before cancellation:");

display(cancelQueue);

cancelEvent(cancelQueue, "E3");

System.out.println();

System.out.println("Queue after cancelling E3:");

display(cancelQueue);

cancelEvent(cancelQueue, "E99");

}

}
```

Test Cases

โปรแกรม **Emergency Call Dispatch Queue** ต้องทดสอบการทำงานในหลายสถานการณ์ เพื่อให้มั่นใจว่า Queue สามารถจัดการ Event ได้ถูกต้องทั้งกรณีปกติและกรณีพิเศษ

ตาราง Test Cases

กรณีทดสอบ	สถานการณ์ทดสอบ	ข้อมูลนำเข้า / การทำงาน	ผลลัพธ์ที่คาดหวัง
TC1	กรณีปกติ	เพิ่ม E1, E2, E3 เข้า FIFO Queue	ประมวลผลตามลำดับ E1 → E2 → E3
TC2	Queue ว่าง	เรียก dequeue() ขณะที่ Queue ว่าง	ไม่เกิดข้อผิดพลาด และ Queue ยังคงว่าง
TC3	มีข้อมูล 1 รายการ	เพิ่ม E1 เพียงรายการเดียว แล้วทำ dequeue()	ประมวลผล E1 และ Queue กลายเป็นว่าง
TC4	Queue ขนาดใหญ่	เพิ่ม Event จำนวน 10,000 รายการ	สามารถจัดเก็บและประมวลผล Event ทั้งหมดได้
TC5	Priority เท่ากัน	เพิ่ม E3 และ E5 ซึ่งมีระดับ Critical เท่ากัน	E3 → E5 เพราะ E3 เข้ามาก่อน
TC6	ยกเลิก Event ที่มีอยู่	ยกเลิก E3 จาก [E1,E2,E3,E4,E5,E6]	E3 ถูกลบออกสำเร็จ
TC7	ยกเลิก Event ที่ไม่มีอยู่	ยกเลิก E99	แสดงข้อความ ไม่พบ Event E99 และข้อมูลเดิมไม่เปลี่ยน
TC8	การจัดลำดับตาม Priority	เพิ่ม Event ที่มี Critical, High, Medium และ Low	ประมวลผลตามลำดับ Critical → High → Medium → Low

รายละเอียดระดับความรุนแรง

ระดับความรุนแรง	ค่า Priority	ความหมาย
Critical	1	ร้ายแรงที่สุด
High	2	ร้ายแรง
Medium	3	ปานกลาง
Low	4	ต่ำ

หมายเหตุ: ใน Priority Queue ค่า Priority ยิ่งน้อยยิ่งมีความสำคัญสูง และถ้า Priority เท่ากัน จะใช้ลำดับการเข้าคิว (Sequence) เป็นตัวตัดสินว่าใครมาก่อน

TC1: Normal Case

Purpose

ทดสอบการทำงานปกติของ FIFO Queue

Input

E1

E2

E3

Queue

[E1, E2, E3]

Operation

DEQUEUE

DEQUEUE

DEQUEUE

Expected Result

$E1 \rightarrow E2 \rightarrow E3$

Explanation

FIFO ใช้หลักการ **First In First Out** ดังนั้น Event ที่เข้ามาก่อนต้องถูกประมวลผลก่อน

TC2: Empty Queue

Purpose

ทดสอบกรณีที่ Queue ไม่มีข้อมูล

Initial Queue

[]

Operation

`queue.poll();`

Expected Result

null

และ Queue ยังคงเป็น

[]

Explanation

เมื่อ Queue ว่าง poll() จะไม่เกิด Error แต่จะคืนค่า null

กรณีนี้ช่วยตรวจสอบว่าโปรแกรมสามารถจัดการ **Empty Queue** ได้อย่างปลอดภัย

TC3: Single Event

Purpose

ทดสอบกรณีที่มี Event เพียงรายการเดียว

Input

E1

Queue

[E1]

Operation

DEQUEUE

Expected Result

DEQUEUE → E1

หลังจากนั้น

[]

Explanation

ตรวจสอบว่า Queue สามารถจัดการกรณีที่มีข้อมูลเพียง 1 รายการได้ถูกต้อง

TC4: Large Queue

Purpose

ทดสอบประสิทธิภาพเมื่อมีข้อมูลจำนวนมาก

Input

10,000 Events

ตัวอย่าง

E1

E2

E3

...

E10000

Operation

ENQUEUE 10,000 Events

DEQUEUE 10,000 Events

Expected Result

All 10,000 Events are processed successfully.

Final Queue = []

Explanation

ใช้ตรวจสอบว่า Queue สามารถรองรับข้อมูลจำนวนมากได้โดยไม่เกิดปัญหาในการจัดเก็บหรือประมวลผล

TC5: Equal Priority

Purpose

ทดสอบกรณีที่ Event มี Priority เท่ากัน

ตัวอย่าง

E3 = Critical = Priority 1

E5 = Critical = Priority 1

แต่

E3 Sequence = 3

E5 Sequence = 5

Queue

E3 → E5

Expected Result

DEQUEUE → E3

DEQUEUE → E5

Explanation

เมื่อ Priority เท่ากัน โปรแกรมจะใช้ sequence เป็นตัวตัดสิน

ดังนั้น Event ที่เข้ามาก่อนจะถูกประมวลผลก่อน

E3 (Sequence 3)



E5 (Sequence 5)

จึงได้

E3 → E5

TC6: Cancel Event

Purpose

ทดสอบการยกเลิก Event ที่มีอยู่ใน Queue

Initial Queue

[E1, E2, E3, E4, E5, E6]

Action

Cancel E3

Expected Result

[E1, E2, E4, E5, E6]

และแสดง

Event cancelled successfully: E3

Explanation

ตรวจสอบว่าโปรแกรมสามารถค้นหาและลบ Event ที่ต้องการออกจาก Queue ได้

TC7: Cancel Non-existing Event

Purpose

ทดสอบการยกเลิก Event ที่ไม่มีอยู่ใน Queue

Initial Queue

[E1, E2, E4, E5, E6]

Action

Cancel E99

แต่ไม่มี E99 ใน Queue

Expected Result

Event not found: E99

Queue ยังคงเป็น

[E1, E2, E4, E5, E6]

Explanation

โปรแกรมต้องไม่ลบข้อมูลอื่น และต้องแจ้งให้ผู้ใช้ทราบว่าไม่พบ Event ที่ต้องการยกเลิก

TC8: Priority Case

Purpose

ทดสอบว่า Priority Queue สามารถจัดลำดับตามความรุนแรงได้ถูกต้อง

Input

E1 = Medium

E2 = Low

E3 = Critical

E4 = High

Priority

Event	Severity	Priority
E1	Medium	3
E2	Low	4
E3	Critical	1
E4	High	2

Expected Result

E3 → E4 → E1 → E2

Explanation

Priority Queue จะเลือก Event ที่มี Priority สูงที่สุดก่อน

Critical → High → Medium → Low

ดังนั้น

E3 → E4 → E1 → E2

สรุป Test Cases

TC1 : Normal Case

TC2 : Empty Queue

TC3 : Single Event

TC4 : Large Queue

TC5 : Equal Priority

TC6 : Cancel Existing Event

TC7 : Cancel Non-existing Event

TC8 : Priority Case

โดย TC5 และ TC7 สำคัญ เพราะเป็นกรณี Edge Case ที่แสดงให้เห็นว่าโปรแกรมไม่ได้ทดสอบแค่การทำงานปกติ แต่รองรับกรณีพิเศษด้วย

Experimental Comparison

การทดลองนี้มีวัตถุประสงค์เพื่อเปรียบเทียบประสิทธิภาพของ **Algorithm A: FIFO Queue** และ **Algorithm B: Priority Queue** เมื่อมีจำนวน Event แตกต่างกัน โดยวัดเวลาการทำงานด้วย `System.nanoTime()`

ขนาดข้อมูลที่ใช้ในการทดลอง

กำหนดจำนวน Event เป็น

- 100 รายการ
- 1,000 รายการ
- 10,000 รายการ
- 50,000 รายการ

แต่ละขนาดข้อมูลทำการทดลอง 5 รอบ แล้วนำเวลาที่ได้อ้อมาคำนวณค่าเฉลี่ย เพื่อให้

จำนวน Event (n)	FIFO รอบที่ 1 (ns)	FIFO รอบที่ 2 (ns)	FIFO รอบที่ 3 (ns)	FIFO รอบที่ 4 (ns)	FIFO รอบที่ 5 (ns)	FIFO เฉลี่ย (ns)	Priority รอบที่ 1 (ns)	Priority รอบที่ 2 (ns)	Priority รอบที่ 3 (ns)	Priority รอบที่ 4 (ns)	Priority รอบที่ 5 (ns)	Priority เฉลี่ย (ns)
100	-	-	-	-	-	-	-	-	-	-	-	-
1,000	-	-	-	-	-	-	-	-	-	-	-	-
10,000	-	-	-	-	-	-	-	-	-	-	-	-
50,000	-	-	-	-	-	-	-	-	-	-	-	-

ผลการทดลองมีความน่าเชื่อถือมากขึ้น

ns = Nanoseconds หรือหน่วยนาโนวินาที

วิธีการทดลอง

1. สร้างข้อมูล

สร้าง Event จำนวน 100, 1,000, 10,000 และ 50,000 รายการ โดยใช้ข้อมูลที่สร้างขึ้นอัตโนมัติ

2. Warm-up

ก่อนเริ่มวัดเวลาจริง ให้โปรแกรมทำงานล่วงหน้าเพื่อให้ Java Virtual Machine (JVM) มีการเตรียมการทำงานก่อนการวัด

3. วัดเวลา

ใช้คำสั่ง

```
long start = System.nanoTime();
```

```
// Operation being tested
```

```
long end = System.nanoTime();
```

```
long time = end - start;
```

เวลาที่ได้คือเวลาที่ใช้ในการทำงานของ Algorithm

4. ทดลอง 5 รอบ

แต่ละขนาดข้อมูลทดลอง 5 รอบ เช่น

Round 1

Round 2

Round 3

Round 4

Round 5

แล้วนำมาหาค่าเฉลี่ย

$$\text{Average} = (\text{Round1} + \text{Round2} + \text{Round3} + \text{Round4} + \text{Round5}) / 5$$

สิ่งที่ต้องวัด

เพื่อให้ตรงกับการวิเคราะห์ Complexity ของงาน สามารถทดลองเปรียบเทียบ Operation หลักได้ดังนี้

Operation	FIFO Queue	Priority Queue
Enqueue	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(\log n)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
Display	$O(n)$	$O(n)$

ผลที่คาดว่าจะพบ

เมื่อจำนวน Event เพิ่มขึ้น จะมีแนวโน้มดังนี้

FIFO Queue

ArrayDeque มีการเพิ่มและนำข้อมูลออกจาก Queue ได้รวดเร็ว

Enqueue $\rightarrow O(1)$

Dequeue $\rightarrow O(1)$

Peek $\rightarrow O(1)$

ดังนั้นเมื่อจำนวน Event เพิ่มขึ้น

100 $\rightarrow$ 1,000 $\rightarrow$ 10,000 $\rightarrow$ 50,000

เวลาของ enqueue และ dequeue ไม่ควรเพิ่มขึ้นอย่างเป็นสัดส่วนกับ n

Priority Queue

PriorityQueue ใช้โครงสร้าง Heap ดังนั้น

Enqueue $\rightarrow O(\log n)$

Dequeue $\rightarrow O(\log n)$

Peek $\rightarrow O(1)$

เมื่อจำนวน Event เพิ่มขึ้น การ enqueue และ dequeue จะมีค่าใช้จ่ายมากกว่า FIFO เนื่องจากต้องรักษาลำดับ

Priority ของ Heap

สรุป Experimental Comparison

หัวข้อ	FIFO Queue	Priority Queue
หลักการ	เข้าก่อนออกก่อน	Priority สูงออกก่อน
โครงสร้างที่ใช้	ArrayDeque	PriorityQueue
Enqueue	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(\log n)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
เหมาะกับ	งานที่ต้องการความยุติธรรมตามลำดับ	งานที่ต้องจัดการเหตุเร่งด่วน
ข้อดี	เร็วและง่าย	จัดการเหตุสำคัญก่อน
ข้อจำกัด	เหตุ Critical อาจต้องรอ	เหตุ Priority ต่ำอาจรอนาน

จากการทดลองพบว่า FIFO Queue มีประสิทธิภาพสูงในการเพิ่มและนำข้อมูลออก เนื่องจาก enqueue และ dequeue มี Time Complexity เท่ากับ $O(1)$ ในขณะที่ Priority Queue มี Time Complexity เท่ากับ $O(\log n)$ สำหรับ enqueue และ dequeue เนื่องจากต้องรักษาลำดับของ Priority อย่างไรก็ตาม Priority Queue เหมาะสมกับระบบ Emergency Call Dispatch มากกว่าในกรณีที่ต้องให้ความสำคัญกับเหตุฉุกเฉินที่มีความรุนแรงสูง ส่วน FIFO เหมาะกับกรณีที่ต้องการรักษาความยุติธรรมตามลำดับเวลาที่รับแจ้งเหตุ

สรุปเปรียบเทียบ Algorithm A และ Algorithm B

หัวข้อ	Algorithm A: FIFO Queue	Algorithm B: Priority Queue
โครงสร้างข้อมูล	ArrayDeque	PriorityQueue
หลักการทำงาน	First In First Out	Priority สูงออกก่อน
การจัดลำดับ	ตามเวลาที่เข้ามา	ตามระดับความรุนแรง
Event ที่มี Priority สูง	ไม่ได้รับสิทธิ์พิเศษ	ได้รับการประมวลผลก่อน
กรณี Priority เท่ากัน	ไม่เกี่ยวข้อง	ใช้ลำดับการเข้ามา (Sequence)
Enqueue	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(\log n)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
Display	$O(n)$	$O(n)$
Space Complexity	$O(n)$	$O(n)$
ข้อดี	ทำงานง่าย รวดเร็ว และยุติธรรมตามลำดับเวลา	สามารถจัดการเหตุการณ์ที่สำคัญก่อน
ข้อจำกัด	เหตุร้ายแรงอาจต้องรอเหตุที่มาก่อน	เหตุ Priority ต่ำอาจต้องรอนาน
เหมาะกับ	งานที่ต้องประมวลผลตามลำดับการเข้ามา	ระบบฉุกเฉินที่ต้องให้ความสำคัญกับเหตุรุนแรง

ตัวอย่างจากโครงการ

Algorithm A: FIFO

E1 → E2 → E3 → E4 → E5 → E6

ประมวลผลตามลำดับเวลาที่รับแจ้ง โดยไม่สนใจระดับความรุนแรง

Algorithm B: Priority Queue

E3 → E5 → E4 → E1 → E6 → E2

ประมวลผลตามระดับความรุนแรง

Critical → High → Medium → Low

และหากมีระดับเดียวกัน จะเลือก Event ที่เข้ามาก่อน

สรุป

Algorithm A (FIFO Queue) เหมาะสำหรับระบบที่ต้องการความยุติธรรมตามลำดับเวลาการเข้ามาของข้อมูล และมีประสิทธิภาพสูงในการ enqueue และ dequeue เนื่องจากมี Time Complexity เท่ากับ $O(1)$ ส่วน

Algorithm B (Priority Queue) เหมาะกับระบบ Emergency Call Dispatch เนื่องจากสามารถให้เหตุการณ์ที่มีความรุนแรงสูงได้รับการประมวลผลก่อน แม้จะเข้ามาทีหลัง แต่มีค่าใช้จ่ายในการ enqueue และ dequeue สูงกว่า FIFO เป็น $O(\log n)$ ดังนั้น สำหรับระบบแจ้งเหตุฉุกเฉิน **Priority Queue มีความเหมาะสมมากกว่า** เพราะความสำคัญของเหตุการณ์ควรถูกนำมาพิจารณาพร้อมกับลำดับเวลาในการรับแจ้งเหตุ